Google Summer of Code 2026 • ML4SCI / DeepLense

Physics-Informed Diffusion Models for Gravitational Lensing Simulation

A Single-Architecture, Controlled-Ablation Study

Applicant	Aditya Singh
Email	aditya26189@gmail.com
GitHub	Aditya26189 (Aditya Singh)
University	IIT Kharagpur — Dual Degree, Mechanical Engineering (Engg. Entrepreneurship); Minor: CS&E; Micro-Spec: AI & Applications | CGPA 8.33
Mentors	Michael Toomey (MIT) • Pranath Reddy (Independent Researcher) • Sergei Gleyzer (University of Alabama) • Hamees Sayed (IIT Madras)
Project Duration	350 hours — Large track (~12 weeks, ~30 hrs/week)
Difficulty	Advanced

Abstract

Strong gravitational lensing is one of the most powerful probes of dark matter substructure. Current state-of-the-art simulators face challenges: the sim-to-real gap causes classifiers trained on simulated lensed images to fail on real observations (Alexander et al., 2021). FlowLensing (Sayed et al., 2025) achieves up to 200× speedup in batch inference over classical ray-tracing (due to GPU batch parallelism; 13.3× per single sample; Table 1), but encodes no lensing physics into its architecture. One of the named mentors for this project, Hamees Sayed (IIT Madras), is the first author of FlowLensing. The FlowLensing paper concludes by identifying as future work: “we aim to weave lensing equations into the model’s architecture for deeper physical fidelity.” This proposal implements that exact direction. The primary deliverable is a physics-valid generator — a two-stage generative pipeline evaluated strictly by its PSNR and AUC performance against FlowLensing. Sim-to-real gap reduction is treated explicitly as a downstream evaluation and ablation target, rather than the core definition of success. Stage 1 (a flow-matching DiT) generates the convergence map $\kappa(\theta)$ conditioned on dark matter class and physical parameters; Stage 2 applies the gravitational lens equation as a native PyTorch differentiable operator. The 350-hour track enables rigorous sequential execution: the two-stage core (Weeks 1–8) is built and gate-checked before two controlled ablations — E(2)-equivariant backbone and auxiliary physics losses — are evaluated on top of it (Weeks 9–18). Deliverables: a working physics-informed simulator with quantified gains over FlowLensing; a five-row ablation table isolating each mechanism’s contribution; and a publicly released diverse lensing dataset.

1. Motivation and Scientific Context

1.1 Strong Gravitational Lensing as a Dark Matter Probe

Strong gravitational lensing occurs when light from a distant source is bent by an intervening massive object, producing Einstein rings, arcs, and multiple images. The precise morphology of these distortions encodes the projected mass distribution of the lens, including its dark matter substructure. Different dark matter candidates predict qualitatively distinct substructure populations: CDM predicts a rich hierarchy of subhalos [1]; ultralight axion dark matter ($m \approx 10^{-23}$ eV) produces vortex-like quantum-pressure-driven defects [1, 13]; and superfluid dark matter models (Berezhiani & Khoury, 2015 [10]) predict exotic vortex substructure arising from dark matter condensation. This phenomenology makes strong lensing an extraordinarily powerful dark matter discriminator — but only at scales that demand fast, high-fidelity simulation.

1.2 The Simulation Bottleneck and the Sim-to-Real Gap

Traditional lensing pipelines (lenstronomy, PyAutoLens) are physically rigorous but prohibitively slow for the scales required by upcoming surveys (Rubin/LSST: tens of thousands of strong lenses [15]). Alexander et al. (2021, arXiv:2112.12121) [3] further demonstrated that classifiers trained on simulated lensing images suffer systematic accuracy loss on real observational data — the sim-to-real domain gap. Sayed et al. (2025, arXiv:2510.07878) [5] addressed the speed bottleneck with FlowLensing, achieving up to 200× speedup in batch inference for intensive dark matter models (13.3× per single sample; FlowLensing Table 1) via a DiT-based flow-matching model. However, FlowLensing is purely data-driven and contains no lensing physics in its architecture. The FlowLensing paper explicitly identifies this as its primary limitation and calls for “weaving lensing equations into the model’s architecture” as future work. This proposal answers that call. This proposal is a direct instance of simulation-based inference [14] applied to dark matter substructure: it advances SBI by hard-coding the forward model as a differentiable operator rather than approximating the simulator from data.

1.3 Physics-Informed Generative Models: Precedents

Physics-informed machine learning has demonstrated that encoding known physical laws as architectural priors produces generators that are not merely statistically realistic but physically consistent. Hoogeboom et al. (2022, arXiv:2203.17003) [6] showed that imposing E(3) equivariance on a diffusion model for molecular generation dramatically improves physical validity. Adam et al. (2022, arXiv:2211.03812) [7] demonstrated score-based generative priors directly in the strong-lensing domain. Most recently, Royo et al. (2026, arXiv:2603.14503) [17] validated the physics-guided diffusion paradigm at the cluster scale, applying a plug-and-play diffusion prior constrained by weak- and strong-lensing observables to reconstruct cluster surface mass density — establishing that explicit physics constraints in diffusion models yield well-calibrated results competitive with expert-tuned methods. Critically, their work addresses the inverse problem (observations → mass map) at the galaxy-cluster scale, and does not target dark matter substructure discrimination. This proposal addresses the forward problem (parameters → physically valid lensed images) at the galaxy-galaxy substructure scale, with the lensing forward model hard-coded as a differentiable operator rather than applied as a posterior sampling constraint. Together, these works motivate the central hypothesis of this proposal: structural encoding of lensing physics — not merely statistical fitting to lensing images — will narrow the sim-to-real gap in ways that purely data-driven approaches cannot.

2. Technical Approach

Architecture overview: The project has one core architecture and two ablation extensions, executed sequentially. The core (Strategy 2) is complete and independently publishable. The extensions (Strategies 1 and 3) are controlled experiments that quantify the marginal contribution of each additional physics-encoding mechanism. This is not scope hedging — it is the correct experimental design for an ablation study.

Phase	Name	What it contributes	Weeks
CORE	**Strategy 2: κ-Map Pipeline**	Hard-codes the lens equation as a differentiable operator. Every generated image is physically valid by construction.	1–6
ABLATION A	Strategy 1: E(2) Equivariance	Tests whether rotational equivariance in the κ-map generator reduces orientation-dependent artefacts and improves power spectrum fidelity.	7–8
ABLATION B	Strategy 3: Physics Losses	Tests whether soft constraints (lens-equation residual, mass-positivity, power-spectrum moment matching) improve training data efficiency and generalisation.	8–9
RELEASE	Dataset + Evaluation + Report	Full ablation table, diverse multi-PSF dataset, public release, and final report.	10–12

2.1 Background: Gravitational Lensing Physics

The lens equation $\beta = \theta - \alpha(\theta)$ relates the true source position β to the observed image position θ via the deflection angle α. The lensing potential $\psi(\theta)$ gives rise to two observables: the convergence $\kappa = \frac{1}{2}\nabla^2\psi$ (dimensionless projected surface mass density) and the complex shear $\gamma = \gamma_1 + i\gamma_2$, where $\gamma_1 = \frac{1}{2}(\psi_{,11} - \psi_{,22})$ and $\gamma_2 = \psi_{,12}$. The convergence and shear are related through the same lensing potential — a hard physical constraint any valid lensed image must satisfy. The dark matter substructure manifests directly in the convergence map: CDM subhalos appear as compact κ spikes; axion vortices appear as ring-like κ depressions. This makes κ the natural intermediate representation for a physics-informed generator. To resolve the mass-sheet degeneracy — the transformation $\kappa \to \lambda\kappa + (1-\lambda)$ produces observationally indistinguishable lensed images for any scalar λ, meaning an unconstrained generator could learn arbitrary κ scales — all lenstronomy-generated κ maps are normalised so that the mean convergence within the Einstein radius θ_E satisfies $\bar{\kappa}(<\theta_E) = 1.0$ before Stage 1 training. This grounds the κ scale physically and reduces scale ambiguity in the generation target.

2.2 Baseline: Reproducing FlowLensing

Before any architectural modifications, I will reproduce the FlowLensing baseline (Sayed et al., 2025) [5] on the three-class DeepLense dataset: no substructure, CDM subhalo (labelled 'sphere' in the dataset, corresponding to spherical NFW subhalo profiles), and axion vortex (a single class — axionic dark matter with $m \approx 10^{-23}$ eV forming vortex defects). The model is a DiT trained via the Lipman et al. (2022) [8] flow-matching objective, conditioned on discrete class label and continuous parameters:

Einstein radius θ_E, source coordinates (x, y), and subhalo mass function slope s (denoted β in FlowLensing; we use s throughout to avoid ambiguity with the source position β in the lens equation). Target metrics to match: PSNR, SSIM, MSE (FlowLensing Table 1), and downstream classification AUC (FlowLensing Table 2). This baseline is the fixed reference point for every ablation in the study. Architecture follows FlowLensing [5] exactly — DiT-B with adaLN conditioning, linear interpolation path, velocity-prediction parameterisation.

2.3 CORE — Strategy 2: Two-Stage κ-Map Pipeline

The two-stage pipeline restructures generation so that the learned component operates on physically interpretable intermediate representations and the final image synthesis is governed by the exact lens equation:

- **Stage 1 (learned):** A flow-matching DiT generates the convergence map κ(θ) conditioned on dark matter class label and continuous physical parameters (θ_E, x, y, s). The training set for Stage 1 is a paired dataset of κ maps generated offline from lenstronomy using the same parameter distributions as the original DeepLense dataset. No ray-tracing is performed during Stage 1 training.
- **Stage 2 (exact physics):** The lens equation is applied as a native PyTorch differentiable operator, mapping κ → lensed image. This is not an external library call; it is a self-contained PyTorch module described in full mathematical detail in Section 2.4 below.

The consequence of this design: every generated image is a physically valid lensing image by construction — not because the network learned to approximate validity from data, but because the lensing forward model is structurally embedded in the generation process. The learned component (Stage 1) only needs to model the distribution of physically plausible κ maps; physical correctness is handled by Stage 2.

Risk mitigation: The DeepLense dataset provides lensed images, not κ maps. Stage 1 will be trained on κ maps re-simulated from the existing parameter distributions using lenstronomy (Week 3). Since all parameters (θ_E, x, y, s) are stored in the dataset metadata, this re-simulation is deterministic and requires no new physical assumptions. Lenstronomy's Python API provides this functionality directly via its LensModel.kappa() method. The initial training set of 10,000 κ maps per class (30,000 total) is smaller than the 89,104-image DeepLense lensed-image dataset, but this is justified: κ maps are structurally simpler targets than full lensed images — they contain no observational noise, no PSF convolution, and no source-lens light superposition — requiring fewer training samples to achieve equivalent distribution coverage. If Stage 1 validation PSNR on held-out κ maps does not converge by epoch 100, the dataset will be expanded to 30,000 images per class using the same lenstronomy pipeline, which takes approximately 6 hours of additional compute on a single T4 GPU. The 10K figure is a testable assumption; the Week 3 power-spectrum consistency check serves as the explicit convergence signal, with 30K expansion as the documented fallback.

2.4 Stage 2: Native PyTorch Differentiable Ray-Tracing

The central technical contribution of Stage 2 is a differentiable implementation of the gravitational lens equation using only native PyTorch autograd primitives. I implement Stage 2 as a self-contained PyTorch module rather than depending on external libraries such as Caustics (Birrer et al.). The Caustics package provides a differentiable lensing implementation, but it is JAX-native. Integrating a JAX-based library with a PyTorch training loop introduces cross-framework latency that would prevent unified gradient flow for end-to-end fine-tuning (Stage 1 gradients cannot propagate through JAX ops), and creates a hard dependency boundary that breaks the DeepLense repository's pure-PyTorch requirement. The self-contained reimplementation below is dependency-free, fully compatible with PyTorch autograd, and allows the Green's-function normalisation, zero-padding strategy, and dimensional conventions to be verified line-by-line against lenstronomy's arcsecond coordinates. The following implementation is fully self-contained.

Deflection Field Computation via 2D FFT

Given the convergence map κ(θ) output by Stage 1, the deflection angle field α(θ) is given by the convolution of κ with the lensing Green's function:

$$\alpha(\theta) = (1/\pi) \int \kappa(\theta') \times (\theta - \theta') / |\theta - \theta'|^2 \; d^2\theta'$$

Computing this directly via spatial convolution is $O(N^4)$ for an N×N grid — computationally prohibitive. Instead, applying the Convolution Theorem, this reduces to pointwise multiplication in Fourier space:

$$\alpha = \mathcal{F}^{-1} (\mathcal{F}(\kappa) \cdot \mathcal{F}(K))$$

where K is the pre-computed Fourier transform of the Green's function kernel. In components, for the x-deflection: $K_x(l) = i \cdot l_x / |l|^2$ where l is the 2D frequency vector (the y-component is analogous). This reduces computational cost to $O(N^2 \log N)$ and, critically, torch.fft.rfft2 and torch.fft.irfft2 are fully differentiable with respect to the input κ — gradients flow through the FFT. To prevent circular convolution artefacts arising from the FFT's implicit periodicity assumption — which would cause mass near one image edge to spuriously deflect rays near the opposite edge, poisoning the autograd backward pass — the κ map is zero-padded to 2N × 2N before applying the Green's function kernel, then cropped back to N × N post-convolution. The padded region represents physically empty space (zero convergence), ensuring the convolution is aperiodic over the actual image domain. The VRAM cost of this padding is negligible: the 2N × 2N intermediate tensors are freed immediately after irfft2, adding approximately 0.05 GB at the working resolution of 64 × 64 pixels (consistent with the FlowLensing dataset specification).

The PyTorch implementation:

```
def kappa_to_deflection(kappa: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
    # kappa: [B, 1, H, W], normalised convergence map from Stage 1
    B, _, H, W = kappa.shape
    # Zero-pad to 2N x 2N to prevent circular convolution artefacts
    H2, W2 = 2 * H, 2 * W
    kappa_pad = F.pad(kappa.squeeze(1), (0, W, 0, H))         # [B, 2H, 2W]
    # Recompute freq grids on padded dimensions
    # Frequency grid convention: d=1.0 (default) returns frequencies in cycles/sample.
     # Frequencies in [0, 1/N, 2/N, ..., -1/N] — dimensionless, cycles per pixel.
# FFT convolution then gives deflection in pixel units.     # Final alpha *= (2.0/W)
converts pixel-unit deflection to [-1,1] grid_sample coords.     # A full-width
deflection (W/2 pixels) maps to 1.0. Verified against lenstronomy     # arcsecond
output in Week 4 unit test (target: pixel error < 1% on 100 test κ maps).     lx2 =
torch.fft.fftfreq(W2, d=1.0, device=kappa.device)
    ly2 = torch.fft.fftfreq(H2, d=1.0, device=kappa.device)
    Ly2, Lx2 = torch.meshgrid(ly2, lx2, indexing='ij')        # [2H, 2W]
    L2p = Lx2**2 + Ly2**2;  L2p[0, 0] = 1.0
    kappa_f = torch.fft.rfft2(kappa_pad, s=(H2, W2))          # [B, 2H, W+1]
    Gx = 1j * Lx2[..., :W2//2+1] / L2p[..., :W2//2+1]
    Gy = 1j * Ly2[..., :W2//2+1] / L2p[..., :W2//2+1]
    # irfft2, then crop back to N x N — padded region discarded
    alpha_x = torch.fft.irfft2(kappa_f*Gx, s=(H2,W2))[:, :H, :W]  # [B,H,W]
    alpha_y = torch.fft.irfft2(kappa_f*Gy, s=(H2,W2))[:, :H, :W]  # [B,H,W]
    # Normalize from pixel-unit deflection to [-1, 1] coordinate system
# required by F.grid_sample. Without this, deflections are ~W/2-fold
# too large and grid_sample returns zeros for all out-of-range coords.
alpha_x = alpha_x * (2.0 / W)
alpha_y = alpha_y * (2.0 / H)
return alpha_x.unsqueeze(1), alpha_y.unsqueeze(1)
```

The factors 2/W and 2/H convert the FFT-domain deflection from pixel-unit coordinates to the [-1, 1] normalized image-plane system expected by F.grid_sample; without this conversion, deflection magnitudes exceed the grid range by a factor of W/2 and the output image is blank. This function is $O(N^2 \log N)$, runs entirely in PyTorch autograd, and adds an estimated ~12 ms of GPU time per batch

on a T4 GPU at the working resolution of 64×64 pixels (to be benchmarked in Week 2 across batch sizes 8/16/32)

Lens Equation and Source Plane Sampling

Once the deflection field is computed, the lens equation maps image plane coordinates θ to source plane coordinates β:

$$\beta = \theta - \alpha(\theta)$$

The lensed image is then constructed by evaluating the source surface brightness I_s at the mapped coordinates:

$$I_obs(\theta) = I_s(\beta(\theta))$$

In PyTorch:

```
def apply_lens_equation(source: torch.Tensor,
                        alpha_x: torch.Tensor,
                        alpha_y: torch.Tensor) -> torch.Tensor:
    B, C, H, W = source.shape
    # Build normalised image-plane grid in [-1, 1]
    grid_y, grid_x = torch.meshgrid(
        torch.linspace(-1, 1, H, device=source.device),
        torch.linspace(-1, 1, W, device=source.device),
        indexing='ij')
    # Map to source plane: beta = theta - alpha
    beta_x = grid_x.unsqueeze(0) - alpha_x.squeeze(1)  # [B, H, W]
    beta_y = grid_y.unsqueeze(0) - alpha_y.squeeze(1)  # [B, H, W]
    # grid_sample expects [B, H, W, 2] in [-1, 1]
    sampling_grid = torch.stack([beta_x, beta_y], dim=-1)
    # F.grid_sample is fully differentiable
    return F.grid_sample(source, sampling_grid,
                        mode='bilinear', align_corners=True)
```

torch.nn.functional.grid_sample provides bilinear interpolation with full autograd support. This completes Stage 2. The combined pipeline (Stage 1 κ generation + Stage 2 ray-tracing) adds no external dependencies and requires no JAX, no Caustics installation, and no cross-framework data transfer.

Source Galaxy Model: Parametric Sérsic Profile

The apply_lens_equation() function takes a source tensor as its first argument. This section defines exactly what that tensor is and where it comes from, since this choice determines whether the pipeline generates physically valid lensed images or merely lensed textures.

There are three candidate options: (a) a parametric Sérsic profile rendered from sampled parameters, (b) a separate learned source generator, or (c) fixed template images from the lenstronomy re-simulation. The choice is (a), for precise reasons:

- **Option (b) is rejected** because a learned source generator inside Stage 2 would introduce a second set of trainable parameters into what is supposed to be an exact physics operator. The core architectural claim of the two-stage pipeline is that Stage 2 contains no learned components — this is what makes it possible to attribute physical consistency to the architecture rather than to the data. A learned source model collapses that distinction and makes the ablation table uninterpretable.
- **Option (c) is rejected** because fixed templates cap morphological diversity. The number of distinct lensed configurations the pipeline can produce is bounded by the size of the template

library, which defeats the purpose of a generative model and reintroduces a sim-to-real gap through a different mechanism.

- **Option (a), the parametric Sérsic profile, is chosen because: (i) lenstronomy uses a Sérsic light profile for source generation (Sayed et al., Section 2), meaning the source morphology space is well-characterised; (ii) at re-simulation time (Week 3), Sérsic parameters (n, r_e, q, φ) will be sampled from standard galaxy morphology priors consistent with lenstronomy's default configuration for the DeepLense dataset — Sérsic index n uniform in [0.5, 4.0], effective radius r_e from a log-normal distribution centred at 0.2 arcseconds calibrated to Euclid-like sources, axis ratio q uniform in [0.3, 1.0], position angle φ uniform in [0, π]. This introduces no assumptions beyond what the dataset already encodes; (iii) a Sérsic profile is a closed-form analytical expression, renderable in ≈15 lines of differentiable PyTorch with zero additional model parameters; and (iv) at inference time, sampling these four parameters from their priors and composing with κ sampling from Stage 1 gives a joint generative model p(κ, source | class, θ_E, x, y, s) that covers the full lensing distribution without any learned source component. A statistical consistency check in Week 3 (power spectrum and Einstein radius distribution comparison between Stage 2 outputs and the original DeepLense images) will confirm prior compatibility. Concrete fallback protocol: if the Einstein radius KS-test p-value drops below 0.05, r_e is rescaled by the ratio of median Einstein radii between Stage 2 outputs and the DeepLense training set. If the Sérsic index distribution diverges by more than 1σ, the n prior is tightened from U(0.5, 4.0) to U(0.5, 2.5) and Stage 2 outputs are re-evaluated before proceeding to Stage 1 training.**

The Sérsic surface brightness profile is defined as:

$$I(r) = I_e \cdot \exp(-b_n \cdot [(r / r_e)^{(1/n)} - 1])$$

The PyTorch implementation:

```
def render_sersic(n, r_e, q, phi, H, W, device) -> torch.Tensor:
    # Build source-plane coordinate grid in [-1, 1]
    gy, gx = torch.meshgrid(torch.linspace(-1,1,H,device=device),
                            torch.linspace(-1,1,W,device=device), indexing='ij')
    # Rotate to position angle phi, apply axis ratio q
    xr =  gx * torch.cos(phi) + gy * torch.sin(phi)
    yr = -gx * torch.sin(phi) + gy * torch.cos(phi)
    r = torch.sqrt(xr**2 + (yr / q)**2).clamp(min=1e-6)
    # Ciotti & Bertin (1999) asymptotic expansion, accurate to < 1e-4 for n > 0.36
    b_n = 2*n - 1/3 + 4/(405*n) + 46/(25515*n**2) + 131/(1148175*n**3)
    I = torch.exp(-b_n * ((r / r_e).pow(1.0 / n) - 1.0))
    return (I / I.sum()).unsqueeze(0).unsqueeze(0)  # [1, 1, H, W], flux-normalised
```

The complete inference recipe is therefore: (1) sample dark matter class and physical parameters (θ_E, x, y, s) from their empirical distributions; (2) sample source Sérsic parameters (n, r_e, q, φ) from the lenstronomy default priors (n ∈ [0.5, 4.0], r_e log-normal at 0.2″, q ∈ [0.3, 1.0], φ ∈ [0, π]) calibrated against the DeepLense image distribution in Week 3; (3) run Stage 1 to generate κ(θ); (4) run kappa_to_deflection() to compute α(θ); (5) run render_sersic() to compute I_s; (6) run apply_lens_equation(I_s, α_x, α_y) to produce I_obs. Steps (3)–(6) are each fully differentiable, so the entire chain from κ to I_obs supports end-to-end gradient flow during optional fine-tuning. The source galaxy model adds zero learned parameters and zero additional training loops to the pipeline.

Memory and VRAM Analysis

Peak VRAM for a batch of 32 images at 64×64 resolution (FlowLensing dataset specification; Sayed et al., 2025):

Operation	Peak VRAM (approx.)	Notes
Stage 1: κ-map DiT forward pass	~6 GB	Matches FlowLensing baseline
Stage 2: FFT deflection + grid_sample	~0.15 GB	Intermediate FFT tensors freed immediately after irfft2
End-to-end fine-tuning (Stage 1 + Stage 2 backward)	~9 GB	Feasible on A100 40GB; marginal on T4 16GB at batch=8

Projected Inference Latency (batch=32, 64×64, T4 GPU)

Pipeline	Latency per batch	vs. lenstronomy
lenstronomy classical	~45 s	1× (baseline)
FlowLensing (Sayed et al., 2025)	~225 ms	~200× faster
This work: Stage 1 + Stage 2	~237 ms	~190× faster

Stage 2 adds ~12 ms per batch to FlowLensing's reported latency. The physics-constrained pipeline preserves >95% of FlowLensing's speedup over classical simulation. Exact figures benchmarked in Week 2.

Training strategy: Stage 1 trains offline on κ maps (no Stage 2 in the loop, VRAM ~6 GB). End-to-end fine-tuning is performed in a separate final pass of Strategy 2 training with gradient checkpointing enabled to bring VRAM below 12 GB on T4. Primary compute is Kaggle free tier (T4, 30 GPU hours/week); Colab Pro+ A100 is used as secondary for the fine-tuning pass only.

2.5 ABLATION A — Strategy 1: E(2)-Equivariant κ Generator

Built on top of the working Strategy 2 foundation, Ablation A reimplements Stage 1 as an E(2)-equivariant U-Net backbone trained via the identical flow-matching objective, replacing the DiT while preserving the conditioning framework, training loss, and evaluation protocol. This is the architecturally correct path: DiT uses multi-head self-attention with absolute positional embeddings, which are not rotation-equivariant by construction — bolting escnn steerable filters onto attention layers does not produce end-to-end equivariance because the positional encoding alone breaks it. An equivariant U-Net (Weiler & Cesa, 2019 [12]) built entirely from escnn steerable convolutional blocks is fully E(2)-equivariant end-to-end. The scientific question being tested is identical: does enforcing rotational equivariance in the κ generator improve the physical validity of generated κ maps (as measured by power spectrum fidelity and Einstein radius distribution), and does this improvement propagate through Stage 2 to improve final lensed image quality? For the full generative flow to be E(2)-equivariant, the velocity field v_θ must satisfy the covariance condition $v_\theta(\varphi \cdot x, t) = D\varphi \cdot v_\theta(x, t)$ for all group elements $\varphi \in E(2)$, where $D\varphi$ is the induced representation on the output space. Under the CFM objective with a straight-line interpolation path $x_t = (1-t)x_0 + tx_1$, this condition is automatically preserved because linear interpolation commutes with linear group actions: $\varphi \cdot x_t = (1-t)(\varphi \cdot x_0) + t(\varphi \cdot x_1)$. The non-trivial requirement is that all conditioning signals must also transform equivariantly. In this implementation: the discrete class label injects as a trivial irrep (invariant scalar); θ_E injects as a trivial irrep (scalar physical quantity invariant under rotation); and the source position (x, y) injects as an irrep-1 vector field on $\mathbb{R}^2$ under the standard 2D rotation representation of O(2). This injection is implemented via the escnn FieldType specification at the conditioning MLP output layer, selecting field type FieldType(gspace, [irrep(1)]) for the (x, y) coordinates — handled in Week 6 field type design. The SIE lens ellipticity position angle and external shear orientation γ_{ext} are also directional quantities that transform under image-plane rotations; both are injected as irrep-1 fields using the same FieldType(gspace, [irrep(1)]) specification. Scalar lens parameters (Einstein radius θ_E, external shear

magnitude $|\gamma_{ext}|$, axis ratio q) inject as trivial irreps. This ensures no conditioning signal breaks E(2)-equivariance at injection.

The E(2) group (2D rotations + reflections) is the correct symmetry group for this application. Lensing configurations have no handedness — a mirrored lens is equally physical — so the full orthogonal group O(2) rather than SO(2) must be used. Equivariance is unit-tested at ≥ 8 rotation angles with residual $\leq 10^{-5}$.

Why this is sequenced second, not first: Redesigning Stage 1 as a full equivariant U-Net requires careful escnn field type selection at each encoder/decoder resolution level, and the correct field types can only be chosen once the baseline Stage 1 DiT's failure modes (which spatial scales matter most, which feature resolutions drive PSNR) are understood from Weeks 3–4. Attempting the backbone swap on an untested baseline conflates two sources of error and makes debugging impossible.

2.6 ABLATION B — Strategy 3: Auxiliary Physics Losses

This ablation adds four physics-based soft constraints to the Strategy 2 training objective, testing whether they improve training data efficiency or generalisation to out-of-distribution lens configurations:

- Lens-equation residual loss (primary): $||I_{obs} - \Phi(\kappa_{pred})||$, where Φ is the Stage 2 operator. A lightweight auxiliary convolutional decoder head (3 conv layers, ~200K parameters) attached to the intermediate feature representations of the Stage 1 denoising backbone — not to its final κ output — predicts an intermediate κ estimate at each denoising step. This enables the residual loss to be computed during training without waiting for the final κ output, providing dense gradient signal throughout the denoising trajectory. The loss uses the flow-matching interpolation estimate $\hat{x}_0(t)$, not the noisy intermediate; this is available at each flow-matching step under the CFM objective.
- **Mass-sheet positivity:** $L_{pos} = \lambda \cdot ||\min(\kappa_{pred}, 0)||_2^2$, penalising negative convergence (non-physical surface mass density).
- **Power-spectrum moment constraint:** $L_{ps} = ||P_{generated}(\ell) - P_{target}(\ell)||_2$ over the angular power spectrum, where P_{target} is estimated from the lenstronomy re-simulated κ training set. This enforces that generated κ maps have the correct statistical structure at all spatial scales. Implementation: a Hann window is applied to each 64×64 map before the 2D DFT to suppress edge-mode mixing artefacts; the resulting power is azimuthally averaged into 10 log-spaced ℓ-bins spanning the Nyquist range. Identical windowing and binning is applied to both generated and target sets, ensuring L_{ps} measures distributional mismatch rather than windowing artefacts.
- **κ-normalisation constraint (mass-sheet degeneracy fix):** $L_{norm} = ||\overline{\kappa}(<\theta_E)_{pred} - 1.0||^2$, penalising deviation of the mean convergence within the predicted Einstein radius annulus from its physical value of 1.0. This soft constraint directly addresses the mass-sheet degeneracy and is complementary to the hard normalisation applied during Stage 1 training.

Note on Kaiser-Squires: The Kaiser-Squires algorithm is a weak-lensing technique designed for statistical shear reconstruction over large sky patches. It is inapplicable here for a precise physical reason: KS assumes the Born approximation and the linear shear-convergence relation $\tilde{\kappa}(l) = |l|^2\tilde{\psi}(l)/2$, which holds only in the weak-lensing regime where $\kappa \ll 1$ and $\gamma \ll 1$ everywhere. In the strong lensing regime — which is the entire domain of this proposal — the shear and convergence fields are non-linear and discontinuous at caustics, violating both the Born approximation and the linear shear-convergence relation. The lens-equation residual loss $L_{res} = ||I_{obs} - \Phi(\kappa_{pred})||^2$ is the principled strong-lensing analogue: it directly penalises violation of the exact lensing forward model without any weak-field approximation. KS is not used.

2.7 Evaluation Suite

Physical Consistency Metrics

- PSNR, SSIM, MSE vs. lenstronomy classical simulations (mirrors FlowLensing Table 1)
- Angular power spectrum P(ℓ): mean and standard deviation per dark matter class, KS-test p-value vs. classical
- Einstein radius distribution: KS-test p-value, generated vs. classical
- Mass-positivity rate: fraction of generated κ maps with min(κ) ≥ 0
- **FID (domain-aware):** computed on ResNet-18 fine-tuned on DeepLense, not Inception-v3. FlowLensing explicitly warns that Inception features "may not be an ideal indicator of astrophysical realism" [5]. Inception Score (IS) is not used.

Downstream Utility Metrics

- ResNet-18 AUC: train on generated data, evaluate on held-out classical simulations (measures simulation fidelity)
- ResNet-18 AUC: train on classical, evaluate on generated (measures distribution coverage)
- domain adaptation (DA): Alexander et al. (2021) [3] protocol — augment real-image fine-tuning with generated samples, measure transfer accuracy improvement
- Ablation table rows: FlowLensing baseline | Strategy 2 (DiT) only | Strategy 2 + non-equivariant U-Net | Strategy 2 + E(2)-equivariant U-Net | Strategy 2 + E(2)-equivariant U-Net + physics losses.

2.8 Codebase Integration

Coding conventions: Black formatter, type hints, NumPy-style docstrings matching existing DeepLense repo style. CI: GitHub Actions with pytest ≥90% coverage gate on all new core modules before PR merge.
Existing modules extended: deeplense/models/flow_lensing/ — FlowLensing baseline reproduced in parallel directory without modifying existing code.
New modules: deeplense/models/flow_lensing_physics/ containing stage1_dit.py (κ-map flow-matching DiT, baseline + Ablation A), stage2_raytracer.py (kappa_to_deflection() + apply_lens_equation()), sersic_source.py (render_sersic() + Sérsic prior sampler), equivariant_unet.py (E(2)-equivariant U-Net for Ablation A), physics_losses.py (L_res, L_pos, L_ps, L_norm for Ablation B). Evaluation: deeplense/eval/physics_metrics.py (power spectrum P(ℓ), Einstein radius KS-test, domain-aware FID). Data: deeplense/data/kappa_simulator.py (lenstronomy κ re-simulation pipeline, multiprocessing.Pool parallelisation).
Target repository: github.com/ML4SCI/DeepLense. Development in personal fork.

3. Implementation Plan and Timeline

350 hours over 12 weeks (~30 hours/week). IIT Kharagpur end-semester examinations conclude late April 2026, leaving the full GSoC coding period (May–October 2026) free. The 350-hour track is justified by the following compute budget: four complete training runs are required (FlowLensing baseline, Stage 1 baseline, Ablation A equivariant U-Net, Ablation B physics losses), each requiring 10–15 GPU-hours on a T4 at 64×64 resolution; the κ dataset re-simulation pipeline (30K images via lenstronomy) adds approximately 5 GPU-hours; evaluation harness runs across five ablation table rows add approximately 8 GPU-hours; debugging, re-training on iteration, and integration work account for the remaining engineering hours. Total GPU-wall-time across the project: approximately 60–70 GPU-hours — well within the Kaggle/Colab Pro+ weekly allocation across 12 weeks. The timeline has hard gates: Ablations A and B only begin if the Strategy 2 foundation passes its Week 5 acceptance criteria (PSNR ≥ baseline, AUC ≥ baseline).

Week	Phase	Tasks	Deliverable	Hours
CB	Community Bonding	Pre-coding orientation: finalize compute allocation (Kaggle T4/Colab Pro+), establish check-in cadence with mentors, and complete high-level codebase orientation. Confirm DeepLense metadata stores complete lens model parameters (ellipticity, position angle, profile type) sufficient for deterministic κ re-simulation.	Codebase familiarity + mentor alignment	–
1	Foundation	Technical work begins: deep literature review (FlowLensing [5], DDPM [4], escnn [12], DiT [9]). Set up DeepLense codebase, lenstronomy environment, wandb logging, and reproducibility seeds.	Env setup + lit notes + mentor call	15
1.5	Foundation	Reproduce FlowLensing baseline DiT on 3-class dataset (no substructure, CDM subhalo, axion vortex). Match reported PSNR, SSIM, MSE from FlowLensing Table 1. Debug training instabilities. Midterm target: training loss curves match FlowLensing reported behaviour. If full metric match not achieved by end of Week 1, proceed to Week 2 with best available checkpoint; metric match reattempted in Week 2 buffer.	Reproduced baseline (PSNR/SSIM/MSE matched)	15
2	Foundation	Complete baseline reproduction: match downstream AUC from FlowLensing Table 2 using ResNet-18 classifier. Generate lenstronomy classical simulation reference set (1,000 images per class). Design native PyTorch Stage 2 module: kappa_to_deflection() (FFT-based, with 2N×2N zero-padding) and apply_lens_equation() (grid_sample).	Stage 2 module (unit tested)	15
2.5	Foundation	Unit test Stage 2 deflection field against lenstronomy numerical output for 10 test κ maps, and confirm ∂L/∂κ autograd flow. Benchmark Stage 2 VRAM and latency across batch sizes 8/16/32 on T4 and A100. Implement gradient checkpointing for end-to-end fine-tuning. Confirm end-to-end VRAM ≤ 12 GB at batch=8 on T4.	Stage 2 perf benchmark	15
3	Foundation	Re-simulate κ map dataset: 10K images per class using lenstronomy.LensModel.kappa() with same (θ_E, x, y, s) parameter distributions as DeepLense. Sérsic source parameters sampled from lenstronomy default priors (n ∈ [0.5, 4.0], r_e log-normal at 0.2″, q ∈ [0.3, 1.0]). Generation parallelised via multiprocessing.Pool across 4 CPU cores; estimated wall-clock time 3–5 hours. Apply κ-normalisation ($\overline{\kappa}$(<θ_E) = 1.0) to all maps before saving. Store as HDF5. Verify Sérsic prior compatibility: if Einstein radius KS-test p-value < 0.05, rescale r_e by ratio of median Einstein radii (Stage 2 vs. DeepLense); if Sérsic index diverges by >1σ, tighten n prior to U(0.5, 2.5) and re-evaluate.	κ map dataset (HDF5)	15
3.5	Foundation	Train Stage 1 (κ-map flow-matching DiT) on κ dataset. Evaluate Stage 1 PSNR/SSIM on held-out κ maps. Run Stage 1 + Stage 2 pipeline end-to-end: generate first lensed images. Visual inspection and basic consistency checks.	Stage 1 trained model + first images	15
4	Foundation	Full evaluation of Strategy 2 pipeline: PSNR, SSIM, MSE, power spectrum P(ℓ), Einstein radius KS test, domain-aware FID. Downstream AUC experiment (ResNet-18, both train/test directions). Mid-project report to mentors.	Strategy 2 full evaluation + mid-report	30
5	Foundation	GATE CHECK: Does Strategy 2 meet acceptance criteria (PSNR ≥ baseline, AUC ≥ baseline)? If yes: proceed to Ablation A. If no: debug and re-train; Ablation A timeline shifts by 1 week. Optional: run end-to-end fine-tuning pass (Stage 1 + Stage 2 jointly).	Gate check + E2E fine-tune	30

6	Extension	Design E(2)-equivariant U-Net architecture for Stage 1: select escnn steerable field types (regular, irrep, trivial) at each encoder/decoder resolution level. Implement steerable convolutional blocks for first two encoder levels. Write E(2) equivariance unit tests: ≥ 8 rotation angles, residual ≤ 10^{-5}. Verify class-label and θ_E conditioning injection is compatible with steerable feature representations. Train non-equivariant U-Net control (same architecture, standard Conv2d) alongside Ablation A to isolate equivariance contribution from architecture contribution.	Equivariant backbone (unit tested)	15
6.5	Extension	Integrate equivariant backbone into Stage 1 training loop. Run full training on κ dataset. First evaluation: κ PSNR/SSIM and power spectrum vs. non-equivariant Stage 1.	Strategy 1 trained κ generator	15
7	Ablation	Run Stage 1 (equivariant) + Stage 2 end-to-end. Full evaluation suite: PSNR, SSIM, power spectrum, AUC. Ablation table row: “Strategy 2 + Equivariance.”	Ablation A results	15
7.5	Ablation	Implement auxiliary decoder head on Stage 1 backbone for κ prediction. Implement lens-equation residual loss L_res = ||I_obs - Φ(κ_pred)||. Implement mass-positivity loss L_pos. Initial training with loss weights λ_res, λ_pos on Strategy 2 baseline.	Physics losses implemente d	15
8	Ablation	2×2 grid search over loss weights (λ_res, λ_pos) — 4 training runs at reduced dataset size (10K images). Select best configuration by validation PSNR.	Loss weight sweep	15
8.5	Ablation	Run full training of Strategy 2 + Ablation B (best loss config). Full evaluation suite. Ablation table rows: “Strategy 2 + Physics Losses” and “All Combined.” (For the All Combined configuration, the auxiliary decoder head will be adapted to handle equivariant feature maps via a group-pooling dimensionality reduction layer before the final projection.)	Ablation B results + full ablation table	15
9	Ablation	Alexander et al. (2021) [3] DA experiment: use generated data from Strategy 2 (best variant) to augment real-image fine-tuning. Measure transfer accuracy improvement vs. no-augmentation baseline.	Sim-to-real gap results	15
9.5	Release	Generate diverse dataset: vary source Sérsic index, lens ellipticity, Einstein radius range. Apply HST, Euclid, and Rubin-like PSFs and noise levels. Rubin variant is a stretch goal — deprioritised if compute is constrained.	Diverse dataset (HST + Euclid)	15
10	Release	Package dataset: HDF5 format, metadata CSV, per-image parameter file, README with generation recipe. All dataset generation scripts included and documented.	Public dataset packaged	15
10.5	Release	Code clean-up: docstrings, type hints, unit tests (≥90% coverage on core modules). Begin integration into ML4SCI/DeepLense repository: open PR, CI/CD passes.	Open PR + CI passing	15
11	Release	Address PR review feedback. Write technical blog post and record 5-minute demo video. Draft final project report.	Blog + demo video	20
12	Release	Final report revision and submission. Mentor sign-off. Respond to any final PR changes. Buffer for any Week 5 gate-check-induced delays.	Merged PR + final report	30

3.1 Risk Register

Risk	Trigger	Probability	Mitigation	De-scope Plan
Stage 1 κ dataset size insufficient	Stage 1 PSNR < baseline by epoch 100	Medium	Expand to 30K images per class (90K total) (takes ~6h compute)	Ablations A and B dropped; baseline only
Stage 1 PSNR fails to meet baseline even after expansion	PSNR < baseline on 30k images	Low	Use standard pre-trained architectures as fallback	Strategy 2 only
Equivariant U-Net (Ablation A) overhead exceeds limits	Training time makes weekly iteration impossible	Medium	Reduce network depth and channel capacity for Ablation A	Replace escnn U-Net with EQ-VAE latent regularization (~50 lines, no custom CUDA)
No real lensing observations with confirmed substructure labels exist for DA evaluation	The field has no labeled real-telescope lensing dataset with confirmed CDM/axion ground truth; all evaluable proxies are simulation-based	High	Implement Alexander et al. UDA protocol exactly: unlabeled target domain uses floating redshift $z \in [0.4, 0.6]$, variable SNR 10–30, and Euclid/HST-like PSF per Table I of ; measure AUC transfer from labeled clean-sim source using ADDA	Report simulated DA results; flag real-data transfer as future work requiring kinematic confirmation
FlowLensing codebase unavailable or broken	Repo missing or requires undocumented dependencies	Low	Re-implement core FlowLensing architecture from paper specifications	Delay Week 1-2; drop stretch goals
Stage 2 FFT calibration error	Pixel error > 1% compared to lenstronomy	Low	SIE analytical deflection fallback	Drop native PyTorch Stage 2, lose end-to-end fine-tuning capability and κ-map generality
Week 5 gate check fails	PSNR < baseline at Week 5	Medium	1-week-per-failure timeline shift policy for architecture debugging	Reduce escnn U-Net channel capacity by 4× and depth by half; re-run equivariance tests.

4. Expected Deliverables

Core Deliverables (unconditional commitments)

- A working two-stage lensing simulator (Stage 1: flow-matching κ generator; Stage 2: native PyTorch differentiable ray-tracer) achieving equal or superior PSNR, SSIM, and downstream AUC to the FlowLensing baseline. Every generated image satisfies the gravitational lens equation by construction. This is the midterm deliverable (Week 5 gate check).
- Clean, documented, unit-tested code integrated into ML4SCI/DeepLense as a merged PR with ≥90% coverage on core modules.
- A technical blog post, 5-minute demo video, and final GSoC project report.

Conditional Deliverables (if Week 5 gate check passes)

Quantitative sim-to-real gap reduction via Alexander et al. (2021) domain adaptation protocol (Week 8).

- FlowLensing baseline | Strategy 2 (DiT) only | Strategy 2 + non-equivariant U-Net | Strategy 2 + E(2)-equivariant U-Net | Strategy 2 + E(2)-equivariant U-Net + physics losses.
- A publicly released dataset of ≥30,000 simulated lensing images across HST and Euclid observational conditions (noise models, PSFs).

Stretch Goals (compute and time permitting)

- Rubin/LSST noise model variant of the diverse dataset. Extension of the κ-map pipeline to continuous dark matter model parameters beyond the three discrete classes.

---

5. Related Work

- **[1] Alexander et al. (2020, arXiv:1909.07346): Deep Learning the Morphology of Dark Matter Substructure — CNN classification of CDM subhalos vs. axion vortex substructure. Establishes the downstream classification protocol this proposal's ablation table replicates.**
- **[2] Alexander et al. (2020, arXiv:2008.12731):** Decoding Dark Matter Substructure without Supervision — unsupervised detection, enabling theory-agnostic inference.
- **[3] Alexander et al. (2021, arXiv:2112.12121): Domain Adaptation for Simulation-Based Dark Matter Searches — quantifies the sim-to-real gap; the DA protocol is the primary benchmark for this proposal.**
- **[4] Ho et al. (2020, arXiv:2006.11239):** Denoising Diffusion Probabilistic Models — foundational DDPM paper establishing the noise schedule and reverse-process parameterisation underlying the DiT backbone used here.
- **[5] Sayed, Reddy, Toomey, Gleyzer (NeurIPS 2025 ML4PS Workshop, arXiv:2510.07878):** FlowLensing — direct predecessor. This proposal implements the exact future-work direction the FlowLensing authors identified: weaving lensing equations into the model's architecture.
- **[6] Hoogeboom, Satorras, Vignac, Welling (2022, arXiv:2203.17003):** Equivariant Diffusion for Molecule Generation in 3D — motivates Ablation A; demonstrates that E(3) equivariance in a diffusion model dramatically improves physical validity of generated structures.
- **[7] Adam et al. (2022, arXiv:2211.03812):** Posterior Samples of Source Galaxies with Score-Based Priors — demonstrates score-based generative priors in the strong-lensing domain, working in the inference direction of source reconstruction rather than the forward simulation direction targeted here; establishes that generative priors over physical quantities improve output quality in lensing tasks.

- **[8] Lipman et al. (2022, arXiv:2210.02747):** Flow Matching for Generative Modeling — introduces the CFM objective used in FlowLensing and reproduced as the baseline here.
- **[9] Peebles & Xie (2023, arXiv:2212.09748, ICCV 2023): Scalable Diffusion Models with Transformers (DiT) — the backbone architecture of FlowLensing and Stage 1.**
- **[10] Berezhiani & Khoury (2015, arXiv:1507.01019):** Theory of Dark Matter Superfluidity — theoretical motivation for vortex-type dark matter substructure in the axion vortex lensing class.
- **[11] Cohen & Welling (2016, arXiv:1602.07576):** Group Equivariant Convolutional Networks — foundational group-equivariance framework underlying escnn.
- **[12] Weiler & Cesa (2019, arXiv:1911.08251):** General E(2)-Equivariant Steerable CNNs (escnn) — provides the E(2) equivariant building blocks used in Ablation A.
- **[13] Hui, Ostriker, Tremaine, Witten (2017, arXiv:1610.08297):** Ultralight Axions in Astronomy — establishes the theoretical basis for ultralight axion dark matter ($m \approx 10^{-23}$ eV) forming quantum-pressure-driven vortex defects; the physical motivation for the axion vortex class in the DeepLense dataset.
- **[14] Cranmer, Brehmer, Louppe (2020, arXiv:1911.01429):** The Frontier of Simulation-Based Inference — establishes the SBI paradigm: generating physically consistent simulations to train classifiers for scientific inference. This proposal is a direct instance of SBI applied to dark matter substructure; the two-stage κ-map pipeline advances SBI by hard-coding the forward model rather than approximating it.
- **[15] Collett, T. E. (2015, arXiv:1507.02093):** The Population of Galaxy-Galaxy Strong Lenses in Forthcoming Optical Imaging Surveys. ApJ 811, 20 — source of the estimate of tens of thousands of strong lenses expected from Rubin/LSST surveys.
- **[16] Ciotti, L. & Bertin, G. (1999):** Analytical properties of the R^(1/m) law. A&A 352, 447 — provides the b_n ≈ 1.9992n − 0.3271 Sérsic approximation used in render_sersic().

---

6. About Me

6.1 Background

I am a third-year student (Batch of 2023) at the Indian Institute of Technology Kharagpur, pursuing a Dual Degree (B.Tech + M.Tech) in Mechanical Engineering with a specialisation in Engineering Entrepreneurship, CGPA 8.33/10. I hold a Minor in Computer Science & Engineering and a Micro-Specialisation in Artificial Intelligence and Applications.

Coursework directly relevant to this proposal: Geometric Methods for Computer Vision (covering Lie groups, group representations, and equivariant architectures — the mathematical foundation of Ablation A); Artificial Intelligence; Probability & Statistics; Linear Algebra. Primary deep learning framework: PyTorch (hard requirement for this project). Additional tools: HuggingFace Transformers, scikit-learn, NumPy, FastAPI, Docker.

6.2 Relevant Technical Experience

Project 1: WorldStrat Ensemble — Scientific Image Super-Resolution (January 2026)

Aditya26189/klymo: A production-grade ensemble system combining Transformer (Swin2SR) and GAN (Real-ESRGAN) architectures to achieve state-of-the-art 4x super-resolution for satellite imagery.

Built a production-grade Swin2SR + Real-ESRGAN ensemble (28.6M + 16.7M parameters) for 4× super-resolution of Sentinel-2 satellite imagery to WorldView-3 resolution. Implemented three adaptive ensemble weighting strategies; validated on 149 samples across three terrain types, achieving PSNR

29.83 dB / SSIM 0.8456 (+0.24 dB over best single model). Built crash-proof inference with checkpoint auto-recovery and GPU OOM fallback.

Direct connection: FlowLensing Table 1 evaluates generated images on PSNR, SSIM, and MSE — exactly the metrics I computed and optimised in WorldStrat. Both tasks require scientific image generation in domains where standard FID is insufficient and domain-specific fidelity metrics are the ground truth. I have direct experience with Swin Transformer architectures (related to the ViT/DiT backbone of FlowLensing), GPU-constrained training pipelines, and the engineering discipline required to produce reproducible PSNR benchmarks.

Project 2: CultRAG at SemEval-2026 Task 7 (Submitted; Under Review)

CultRAG/BLEnD-CultRAG: BLEnD-CultureRAG is a retrieval-augmented cultural reasoning system. This system evaluates the cultural awareness of language models by selecting culturally appropriate answers to English multiple-choice questions.

First-authored a research paper on hybrid sparse-dense retrieval for cultural QA, evaluating a 6-phase pipeline on 47K queries with 2-GPU tensor-parallel inference (Llama-3.1-8B, 4-bit NF4 quantisation). The paper includes a statistically rigorous ablation study with McNemar's testing and per-locale breakdown of component contributions.

Direct connection: This proposal's core deliverable is a controlled ablation table with five rows, each row requiring independent training runs and statistical comparison. I have already executed this experimental design at publication quality. I know how to isolate component contributions, report variance, and defend the methodology under peer review.

Project 3: ML4SCI DeepLense Common Test — Equivariant 3-Class Gravitational Lensing Classification (March 2026)

Architecture: C8LensNet (673K parameters) — cyclic-group C8 equivariant CNN using escnn steerable convolutions on rot2dOnR2(N=8), with GroupPooling to produce rotation-invariant features. Compared against ResNet-18 (11.2M parameters) with RandomRotation(180°) augmentation as baseline.

Results on held-out test set:

```
no_sub  AUC: 0.9814 (C8) / 0.9886 (R18)
sphere  AUC: 0.9551 (C8) / 0.9683 (R18)
vort    AUC: 0.9781 (C8) / 0.9824 (R18)
Macro   AUC: 0.9715 (C8) / 0.9798 (R18)
```

C8LensNet achieves macro AUC 0.9715 — within 0.83 pp of ResNet-18 — at 16.6× fewer parameters. Key finding: vort AUC 0.9781 with default argmax accuracy 67.9% is a calibration gap, not a representation failure — post-hoc threshold scaling recovers vort accuracy to 94.2% with zero AUC change, confirming the equivariant representation already contains full discriminative signal. What didn't work: direct argmax on the raw softmax underperforms at the narrow vort/sphere decision boundary due to systematic underconfidence in the GroupPool invariant space.

Note: the classification AUC gap (C8 0.9715 vs ResNet-18 0.9798) does not predict generation performance under equivariance. Classification requires GroupPooling, which intentionally discards all orientational information to produce rotation-invariant predictions — equivariance is a lossy compression in that context. Generation must preserve orientational structure, not discard it; the equivariant backbone constrains the generator to produce κ maps that are physically consistent across all rotations, which is exactly what the gravitational lensing symmetry group demands.

6.3 Why This Project, and Why the 350-Hour Track

The FlowLensing paper ends with a precise research question I can answer: “we aim to weave lensing equations into the model’s architecture.” Hamees Sayed, the first author of FlowLensing, is one of the named mentors for this project. This is not a coincidence — this proposal is a direct response to the future-work direction Sayed identified in his own paper. The two-stage κ-map pipeline with native PyTorch ray-tracing is the exact architectural answer to that question — hard-coding the forward model as a differentiable operator is not a heuristic, it is the correct physics-informed generative design. I chose the 350-hour track because the 175-hour version of this proposal was architecturally overextended: it promised three independent architectural overhauls that would each require 175 hours to execute with the rigour this study demands. The 350-hour track allows me to execute the core pipeline completely, then build the ablations on top of a working foundation — which is the correct experimental design for a study that aims to isolate the contribution of each physics-encoding mechanism.

The Geometric Methods for Computer Vision coursework I completed at IIT Kharagpur covers the Lie group theory that makes E(2)-equivariant denoising principled rather than heuristically motivated. The PSNR/SSIM benchmarking discipline I developed in WorldStrat maps directly onto the FlowLensing evaluation protocol. And the ablation study methodology I applied in the CultRAG paper is the same methodology this project’s ablation table requires. I am not starting from zero.

6.4 Availability and Communication

I can commit 30–35 hours per week for the full GSoC coding period (May–August 2026). IIT Kharagpur end-semester examinations conclude late April 2026; the entire coding window is free of academic obligations. I am in IST (UTC+5:30) and will schedule weekly video calls at times convenient for US-Eastern (MIT, U. Florida, U. Alabama) timezones. I will send weekly written progress updates and flag blockers immediately.

GPU Compute. Primary compute: Kaggle free tier (30 GPU-hours/week, T4/P100). Sessions have a 12-hour limit; all training runs checkpoint every epoch to a persistent Kaggle Dataset (ddpm_latest.pt), enabling seamless resumption across sessions with no progress loss — tested and verified across 300-epoch runs. Secondary: Google Colab Pro+ (A100 40 GB, used for end-to-end fine-tuning passes that exceed Kaggle’s VRAM). All training runs are checkpointed every 10 epochs using wandb artifacts saved to a persistent Kaggle dataset, so no progress is lost to session interruptions. If ML4SCI provides a compute allocation (as some GSoC organisations do), I will request it at acceptance and use it as the primary resource for Ablation A and B runs, which are the most VRAM-intensive phases. Based on comparable DiT workloads at 64×64 resolution (matching the FlowLensing dataset), Stage 1 training throughput on a T4 GPU is estimated at 2–3 minutes per epoch at batch size 32. A full 10K×3-class Stage 1 training run is projected at approximately 10–12 hours end-to-end — fitting comfortably within the standard weekly Kaggle allocation.

References

1. [1] S. Alexander, S. Gleyzer, E. McDonough, M. W. Toomey, E. Usai. Deep Learning the Morphology of Dark Matter Substructure. Astrophys. J. 893, 15 (2020). arXiv:1909.07346.
2. [2] S. Alexander, S. Gleyzer, H. Parul, P. Reddy, M. W. Toomey, E. Usai, R. Von Klar. Decoding Dark Matter Substructure without Supervision. arXiv:2008.12731 (2020).
3. [3] S. Alexander, S. Gleyzer, P. Reddy, M. Tidball, M. W. Toomey. Domain Adaptation for Simulation-Based Dark Matter Searches Using Strong Gravitational Lensing. arXiv:2112.12121 (2021).
4. [4] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. NeurIPS 2020. arXiv:2006.11239.
5. [5] H. Sayed, P. Reddy, M. W. Toomey, S. Gleyzer. FlowLensing: Simulating Gravitational Lensing with Flow Matching. NeurIPS 2025. arXiv:2510.07878.

6. [6] E. Hoogeboom, V. G. Satorras, C. Vignac, M. Welling. Equivariant Diffusion for Molecule Generation in 3D. ICML 2022. arXiv:2203.17003.
7. [7] A. Adam, A. Coogan, N. Malkin, R. Legin, L. Perreault-Levasseur, Y. D. Hezaveh, Y. Bengio. Posterior Samples of Source Galaxies in Strong Gravitational Lenses with Score-Based Priors. arXiv:2211.03812 (2022).
8. [8] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, M. Le. Flow Matching for Generative Modeling. ICLR 2023. arXiv:2210.02747.
9. [9] W. S. Peebles, S. Xie. Scalable Diffusion Models with Transformers (DiT). ICCV 2023. arXiv:2212.09748.
10. [10] L. Berezhiani, J. Khoury. Theory of Dark Matter Superfluidity. Phys. Rev. D 92, 103510 (2015). arXiv:1507.01019.
11. [11] T. S. Cohen, M. Welling. Group Equivariant Convolutional Networks. ICML 2016. arXiv:1602.07576.
12. [12] M. Weiler, G. Cesa. General E(2)-Equivariant Steerable CNNs. NeurIPS 2019. arXiv:1911.08251.
13. [13] L. Hui, J. P. Ostriker, S. Tremaine, E. Witten. Ultralight axions in astronomy. Phys. Rev. D 95, 043541 (2017). arXiv:1610.08297.
14. [14] K. Cranmer, J. Brehmer, G. Louppe. The Frontier of Simulation-Based Inference. PNAS 117 (30), 2020. arXiv:1911.01429.
15. [15] T. E. Collett. The Population of Galaxy-Galaxy Strong Lenses in Forthcoming Optical Imaging Surveys. ApJ 811, 20 (2015). arXiv:1507.02093.
16. [16] L. Ciotti, G. Bertin. Analytical properties of the R^(1/m) law. A&A 352, 447 (1999).
17. [17] D. Royo, B. Zhao, A. Muñoz, D. Gutierrez, K. L. Bouman. Mapping Dark-Matter Clusters via Physics-Guided Diffusion Models. arXiv:2603.14503 (2026).

Submitted to Google Summer of Code 2026 under the ML4SCI organisation.